Агентный ИИ на Kubernetes

kind + kagent + локальный Ollama

Учебный проект: полностью локальный запуск ИИ-агентов на Kubernetes

О чём этот проект

Проблема: запустить «агентный ИИ» (agentic AI) на Kubernetes **полностью локально** — без облачных LLM и без API-ключей.

Решение — три компонента:

Компонент	Роль
kind	Локальный кластер Kubernetes в Docker
kagent	Kubernetes-нативный фреймворк для ИИ-агентов (CNCF Sandbox)
Ollama	Локальный LLM-сервер внутри кластера

В результате мы получаем готового агента `k8s-expert` — Kubernetes/DevOps-эксперта, с которым можно общаться через CLI или веб-интерфейс.

Ключевые понятия (обзор)

- **Agentic AI (агентный ИИ)** — ИИ, который не просто отвечает, а **самостоятельно вызывает инструменты** и выполняет действия.
- **Kubernetes-native** — агенты описываются как обычные ресурсы Kubernetes (CRD), а не как отдельные приложения.
- **CRD (Custom Resource Definition)** — расширение API Kubernetes собственными типами ресурсов.
- **LLM (Large Language Model)** — большая языковая модель; здесь — локальная `qwen3:8b`.
- **Local inference** — запуск модели на своём железе, без облака.
- **Tool/Function calling** — способность модели вызывать внешние функции/инструменты.

Агентный ИИ (Agentic AI)

Отличие от обычного чат-бота:

- **Чат-бот:** вопрос → ответ (текст в текст).
- **Агент:** рассуждение → **выбор инструмента** → действие → наблюдение → снова рассуждение (цикл).

```
Пользователь → Агент → LLM (решает, что делать)
                 ↑          │
                 └── результат инструмента ← инструмент (kubectl, логи, ...)
```

Требование: модель обязана поддерживать **tool/function calling**, иначе агент не сможет вызвать ни один инструмент.

kagent — главный герой (CNCF Sandbox)

kagent — Kubernetes-native фреймворк для создания и запуска ИИ-агентов.

- Открытый исходный код, проект **CNCF Sandbox**.
- Превращает агентов в **CRD** (ресурсы Kubernetes).
- Управляет жизненным циклом агента через **controller** (оператор-паттерн).
- Поддерживает разные **model providers** (Ollama, OpenAI и др.).
- Даёт инструменты через **MCP-сервер**, общение — по протоколу **A2A**.

> Вместо отдельного сервиса — агент описывается в YAML, как обычный ресурс кластера.

CRD-модель kagent

kagent моделирует агентов тремя основными ресурсами:

CRD	Назначение	Аналогия
Harness	инфраструктура выполнения + политики	«рантайм»
AgentTemplate	промпт + модель + инструменты	«образ/шаблон»
AgentInstance	один запущенный разговор	«под/контейнер»

Дополнительно:

- **ModelConfig —** конфигурация LLM-провайдера.
- **WorkerPool —** пул воркеров для выполнения агентов.

Harness — инфраструктура выполнения

`Harness` объявляет **переиспользуемый рантайм + политику**:

```
spec:
  kagent: {}                          # нативный runtime-адаптер kagent
  workload:
    image: .../golang-adk@sha256:...   # образ закреплён по digest
  substrate:
    workerPoolRef: { name: kagent-default }
    snapshotPolicy:
      location: s3://ate-snapshots/kagent/
  allowedAgentTemplates:
    selector: { matchLabels: { kagent.dev/harness: kagent } }
```

- `kagent: {}` — нативный рантайм.
- `substrate.workerPoolRef` — **где** выполняются агенты (gVisor-воркеры).
- `snapshotPolicy` — куда сохранять снимки состояния.
- `allowedAgentTemplates` — **кого** Harness допускает по меткам.

AgentTemplate — поведение агента

`AgentTemplate` описывает **переносимое поведение** (промпт + модель + инструменты):

```
metadata:
  name: k8s-expert
  labels: { kagent.dev/harness: kagent }   # связь с Harness
spec:
  modelConfig: { name: default-model-config }
  systemPrompt: |
    You are a senior platform/DevOps engineer and Kubernetes expert...
  tools:
    - mcp:
        server: { kind: RemoteMCPServer, name: kagent-tool-server }
        tools: [k8s_apply_manifest, k8s_get_resources,
                k8s_describe_resource, k8s_get_pod_logs,
                k8s_get_events, k8s_rollout, k8s_scale, ...]
```

Системный промпт объясняет агенту, **какие инструменты есть и когда их применять**.

AgentInstance — живой разговор

`AgentInstance` = **один запущенный разговор** (экземпляр агента).

```
# создать экземпляр на Harness kagent из шаблона k8s-expert
kagent create agent-instance --harness kagent --agent-template k8s-expert

# отправить задачу и получить ответ (по A2A)
kagent invoke --agent-instance <id> --task "Why does my Pod stay in Pending?"
```

- Создание AgentInstance запускает **Actor** на WorkerPool.
- CLI возвращает управление, когда экземпляр достигает состояния **READY**.
- `invoke` отправляет задачу; `--stream` стримит ответ по мере генерации.

ModelConfig и провайдер Ollama

`ModelConfig` описывает **LLM-провайдера**, к которому обращается агент.

```
providers:
  default: ollama
  ollama:
    provider: Ollama
    model: "qwen3:8b"
    config:
      host: http://ollama.ollama.svc.cluster.local:11434
      options:
        num_ctx: "32768"
```

- Чарт генерирует ModelConfig **`default-model-config`**.
- `host` — **внутрикластерный** Service Ollama.
- `num_ctx` — размер контекстного окна модели.

> Почему в кластере? Песочницы агентов имеют контролируемый egress — они видят внутрикластерный DNS, но **не** Ollama на ноутбуке.

Agent Substrate — движок выполнения

Agent Substrate — исполняющий движок kagent 1.x.

- Запускает **Actors** — песочницы (по одной на ход агента).
- Между ходами Actor **приостанавливается**, состояние — в снимках (snapshots).
- Снимки пишутся в объектное хранилище (`rustfs`, S3-совместимое).
- Компоненты (namespace `ate-system`): `ate-api-server`, `ate-controller`, `atelet`, `atenet-router`, `rustfs`, `postgres`.
- Аутентификация компонентов — **взаимный TLS (mTLS)**.

```
ход (turn) → запуск Actor → выполнение → снимок состояния → пауза
```

gVisor-песочница и WorkerPool

Каждый агент выполняется **изолированно**:

- **gVisor** — рантайм, перехватывающий системные вызовы (изоляция от хоста).
- `sandboxClass: gvisor` — обязателен для kagent ActorTemplates.
- Образ воркера: `ateom-gvisor`.

WorkerPool — пул воркеров, где выполняются ходы агентов:

- Один воркер = **один Actor** одновременно.
- Между ходами воркеры освобождаются.
- Масштабирование:

```
kubectl scale workerpool kagent-default -n kagent --replicas=2
```

Ollama — локальная LLM

Ollama запускает LLM внутри кластера (без облака и API-ключей).

Манифест (`manifests/ollama/ollama.yaml`):

- **Namespace** `ollama`, **Deployment** (1 реплика), **Service** :11434.
- **PVC** `ollama-models` → `/root/.ollama` (модель переживает рестарт пода).
- `OLLAMA_HOST=0.0.0.0`, readiness-проба по TCP-порту 11434.
- `OLLAMA_KEEP_ALIVE=5m` — выгрузка неиспользуемых моделей из RAM.

Модель	Размер	Tool calling
`qwen3:8b`	~4.9 ГБ	✅ сильный (по умолчанию)
`qwen3:4b`	~2.4 ГБ	✅ хороший
`llama3.2:3b`	~2 ГБ	⚠️ базовый

kind — локальный кластер Kubernetes

kind (Kubernetes IN Docker) — настоящий кластер K8s в Docker на ноутбуке.

Требования и особенности (`config/kind-cluster.yaml`):

- **Kubernetes ≥ 1.37** — требование Agent Substrate.
- `runtimeConfig: "certificates.k8s.io/v1beta1": "true"` — нужен Substrate.
- `networking.dnsSearch: []` — не наследовать DNS search-домены хоста (иначе wildcard-записи ломают разрешение `registry.ollama.ai` и TLS).

```
sed 's/__CLUSTER_NAME__/kagent/' config/kind-cluster.yaml \
  | kind create cluster --image kindest/node:v1.37.0 --config=-
```

MCP-сервер инструментов

MCP (Model Context Protocol) — открытый протокол доступа LLM к инструментам и данным.

kagent поставляет встроенный **MCP tool server** (`kagent-tool-server`):

- Инструменты: Kubernetes, Helm, Argo, Prometheus, Grafana, Cilium.
- Мы подключаем только **Kubernetes-поднабор**:
  - `k8s_apply_manifest`, `k8s_create_resource`, `k8s_generate_resource`
  - `k8s_get_resources`, `k8s_get_resource_yaml`, `k8s_describe_resource`
  - `k8s_get_events`, `k8s_get_pod_logs`
  - `k8s_rollout`, `k8s_scale`, `k8s_delete_resource`
- Tool server работает с **cluster-admin RBAC**.

Протокол A2A (Agent-to-Agent)

A2A — открытый протокол взаимодействия агентов и клиентов.

- CLI и веб-UI общаются с контроллером по **A2A / gRPC**.
- Ответ агента **стримится** обратно по мере генерации.
- Порты: контроллер **8083**, веб-UI **8082**.

```
kagent CLI ─┐
            ├─→ kagent-controller (8083, A2A/gRPC) ─→ AgentInstance
browser ────┘          (веб-UI на 8082)
```

CLI сам делает `kubectl port-forward` к сервису `kagent-controller`, если порт свободен.

Tool/Function calling

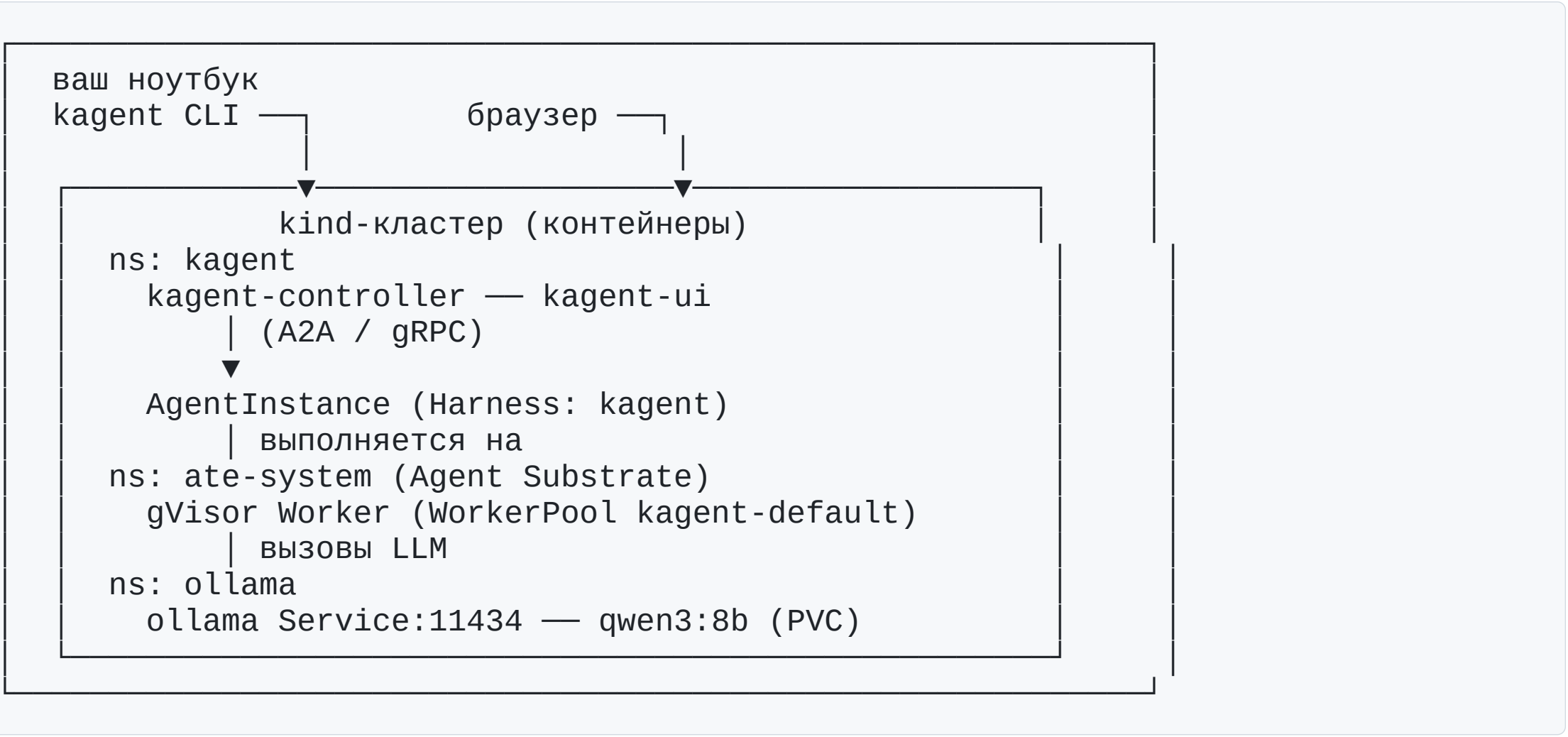
Способность модели генерировать **структурированные вызовы функций**.

```
LLM: "нужно посмотреть логи пода" → вызов k8s_get_pod_logs(pod=..., ns=...)
runtime выполняет инструмент → результат возвращается модели → LLM отвечает
```

- Модель возвращает **имя функции + аргументы** (JSON), рантайм исполняет.
- Без поддержки function calling агент **не сможет** пользоваться инструментами:
  - подключится, но упадёт при первом вызове инструмента;
  - либо начнёт галлюцинировать ответы.

> Поэтому выбрана `qwen3:8b` — сильная поддержка tool calling.

Архитектура (единая схема)



Как это работает (поток запроса)

1. Пользователь задаёт вопрос агенту `k8s-expert`.
2. Контроллер kagent находит **Worker** в WorkerPool.
3. Стартует **Actor** (gVisor-песочница) с конфигурацией модели и промптом.
4. Рантайм отправляет запрос в **Ollama** (внутрикластерный Service).
5. Модель при необходимости вызывает **инструмент** через MCP-сервер (kubectl).
6. Результат инструмента возвращается модели — цикл до готового ответа.
7. Ответ **стримится** пользователю по **A2A**.

> Агенты выполняются в песочницах с контролируемым egress: доступен внутрикластерный DNS, но не ресурсы ноутбука.

Identity bootstrap (mTLS)

Компоненты Agent Substrate аутентифицируются **взаимным TLS** — материал идентичности создаётся вручную (`kubectl-ate`):

- CA-пулы: `service-dns-ca-pool`, `pod-identity-ca-pool`.
- Идентичность актора: `actor-id-jwt-pool` + `actor-id-ca-pool`.
- `egress-mitm-ca-pool` — сертификаты для перехвата HTTPS в egress-шлюзе.
- Секрет `actor-id-ca-certs` — корневой сертификат актора для API-сервера.
- `ate-api-authentication` — приём токенов Kubernetes ServiceAccount; **issuer берётся из OIDC-документа кластера** (не хардкодится).

Затем: `helm upgrade substrate --reuse-values --wait` (монтирует идентичность).

Быстрый старт

```
# 1. проверить / установить инструменты
make prereqs

# 2. поднять весь стек (первый запуск качает образы и модель)
make up

# 3. поговорить с агентом
make chat Q="Why does my Pod stay in Pending state?"

# 4. (опционально) открыть веб-UI -> http://localhost:8082
make ui

# 5. удалить всё
make down
```

Модель настраивается: `make up OLLAMA_MODEL=qwen3:4b`

Структура репозитория

```
├── Makefile                        # цели: prereqs, up, chat, ui, down, ...
├── config/
│   ├── kind-cluster.yaml           # конфиг кластера (k8s 1.37)
│   └── kagent-values.yaml          # Helm-значения (ollama + substrate)
├── manifests/
│   ├── ollama/ollama.yaml          # Ollama Deployment + Service + PVC
│   └── kagent/
│       ├── harness.yaml            # Harness (рантайм + WorkerPool)
│       └── agent-template.yaml     # агент "k8s-expert"
├── scripts/
│   ├── lib.sh                      # версии, дефолты, хелперы
│   ├── up.sh / chat.sh / ui.sh / status.sh / down.sh
└── docs/TUTORIAL.md                # пошаговый разбор
```

Версии и требования

Компонент	Версия
kagent	`1.0.0-alpha3`
Agent Substrate	`0.2.0-beta5`
runtime-образ	`golang-adk` (по digest)
Ollama-модель	`qwen3:8b` (настраиваемая)
Kubernetes (kind)	`1.37.0`

Аппаратные требования: 16 ГБ RAM (в идеале 32 ГБ), ~30 ГБ диска.

Типичные проблемы и решения

- **kind + ZFS** → собрать нод-образ с ZFS-инструментами (`build-node-image.sh`).
- **x509 / wrong certificate** → DNS search-домены хоста: `dnsSearch: []` в конфиге.
- **`kubectl ate --help` не работает** → `make prereqs`.
- **AgentInstance в `UNKNOWN`** → смотреть `status.harnesses[].conditions`.
- **Агент не отвечает вовремя** → масштабировать WorkerPool.
- **Ollama без памяти** → снизить лимиты памяти или выбрать модель меньше.
- **Модель не использует инструменты** → нет поддержки function calling.

Выводы

- **Полностью локальный агентный ИИ на Kubernetes —** без облака и API-ключей.
- **kagent** превращает агентов в CRD: декларативность, GitOps, RBAC, наблюдаемость.
- **Agent Substrate** изолирует выполнение в gVisor-песочницах со снимками состояния.
- **Ollama** даёт локальную модель с поддержкой tool calling.
- Агент `k8s-expert` реально управляет Kubernetes через MCP-инструменты.

Куда развивать: больше инструментов, несколько моделей, масштабирование WorkerPool, OpenTelemetry-трассировка.

Спасибо за внимание!

Вопросы?

Ссылки

- kagent: https://kagent.dev/ · github.com/kagent-dev/kagent
- Agent Substrate: github.com/kagent-dev/substrate
- kind: https://kind.sigs.k8s.io/ · Ollama: https://ollama.com/